

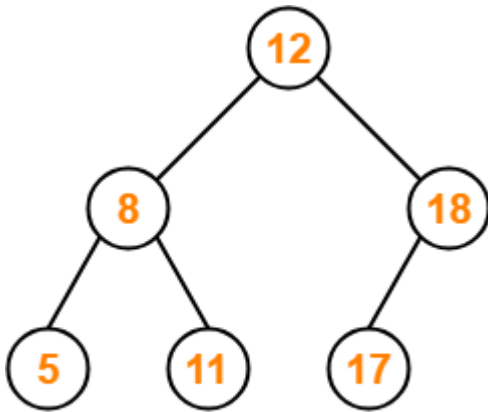
AVL Tree | AVL Tree Example | AVL Tree Rotation

AVL Tree-

- AVL trees are special kind of binary search trees.
- In AVL trees, height of left subtree and right subtree of every node differs by at most one.
- AVL trees are also called as **self-balancing binary search trees**.

Example-

Following tree is an example of AVL tree-

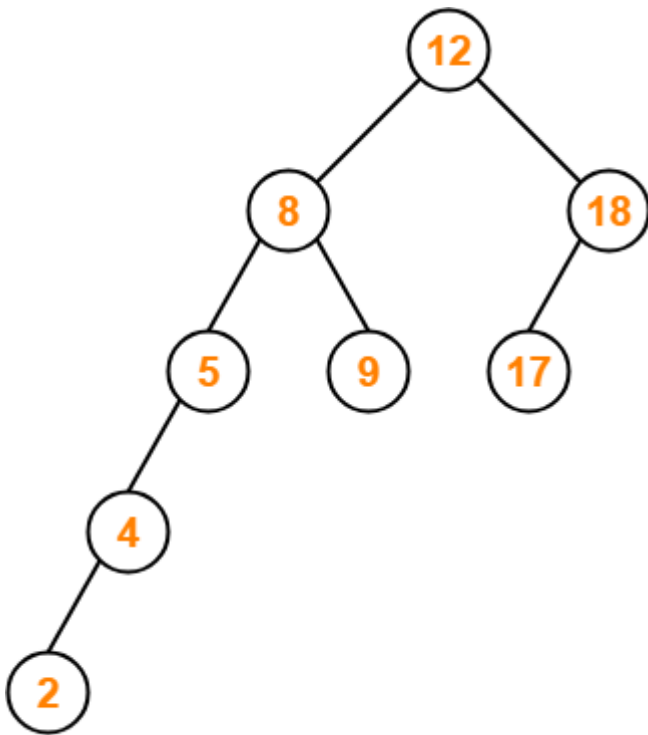


AVL Tree Example

This tree is an AVL tree because-

- It is a binary search tree.
- The difference between height of left subtree and right subtree of every node is at most one.

Following tree is not an example of AVL Tree-



Not an AVL Tree

This tree is not an AVL tree because-

- The difference between height of left subtree and right subtree of root node = $4 - 2 = 2$.
- This difference is greater than one.

Balance Factor-

In AVL tree,

- Balance factor is defined for every node.
- Balance factor of a node = Height of its left subtree – Height of its right subtree

In AVL tree,

Balance factor of every node is either 0 or 1 or -1.

AVL Tree Operations-

Like **BST Operations**, commonly performed operations on AVL tree are-

1. Search Operation
2. Insertion Operation
3. Deletion Operation

Also Read- Insertion in AVL Tree

After performing any operation on AVL tree, the balance factor of each node is checked.

There are following two cases possible-

Case-01:

- After the operation, the balance factor of each node is either 0 or 1 or -1.
- In this case, the AVL tree is considered to be balanced.
- The operation is concluded.

Case-02:

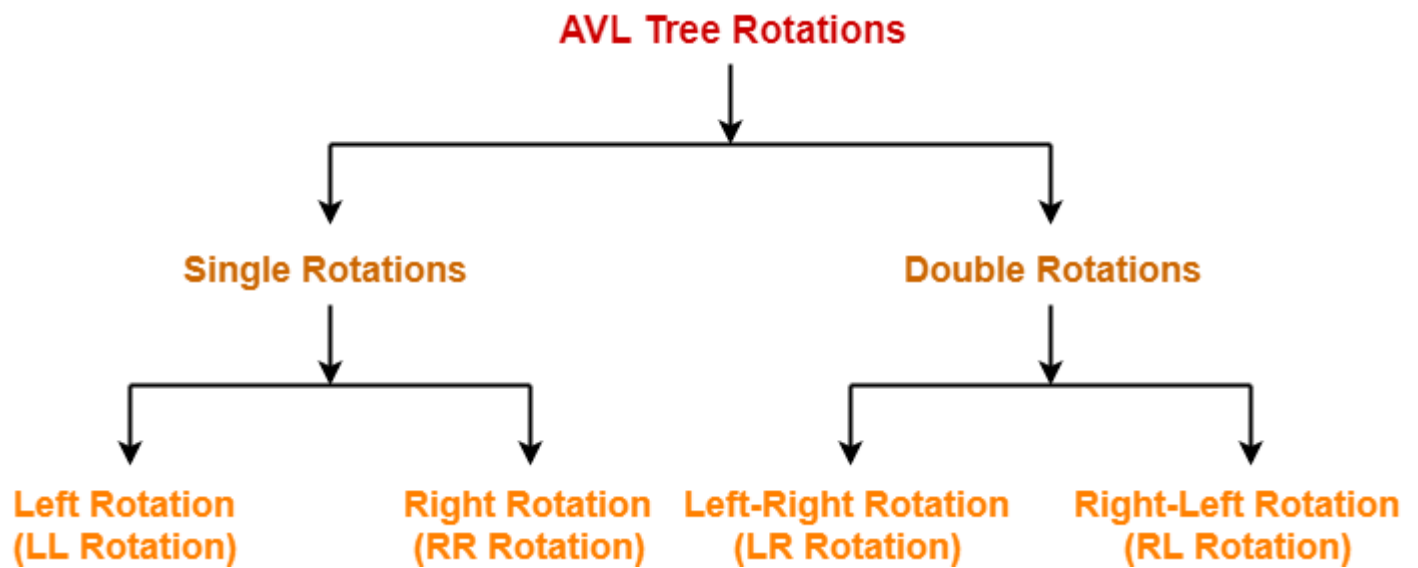
- After the operation, the balance factor of at least one node is not 0 or 1 or -1.
- In this case, the AVL tree is considered to be imbalanced.
- Rotations are then performed to balance the tree.

AVL Tree Rotations-

Rotation is the process of moving the nodes to make tree balanced.

Kinds of Rotations-

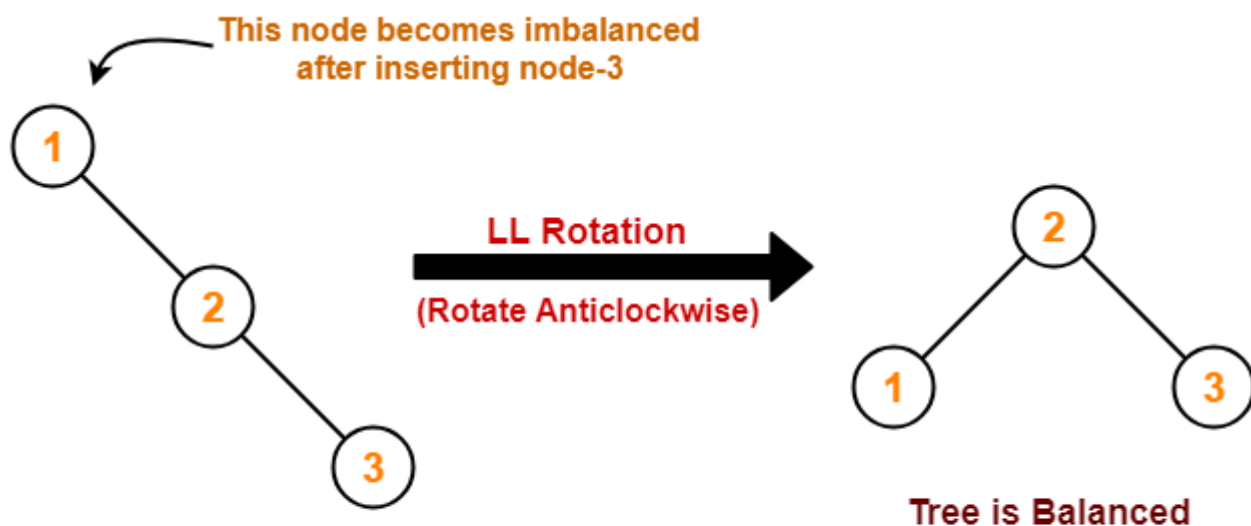
There are 4 kinds of rotations possible in AVL Trees-



1. Left Rotation (LL Rotation)
2. Right Rotation (RR Rotation)
3. Left-Right Rotation (LR Rotation)
4. Right-Left Rotation (RL Rotation)

Cases Of Imbalance And Their Balancing Using Rotation Operations-

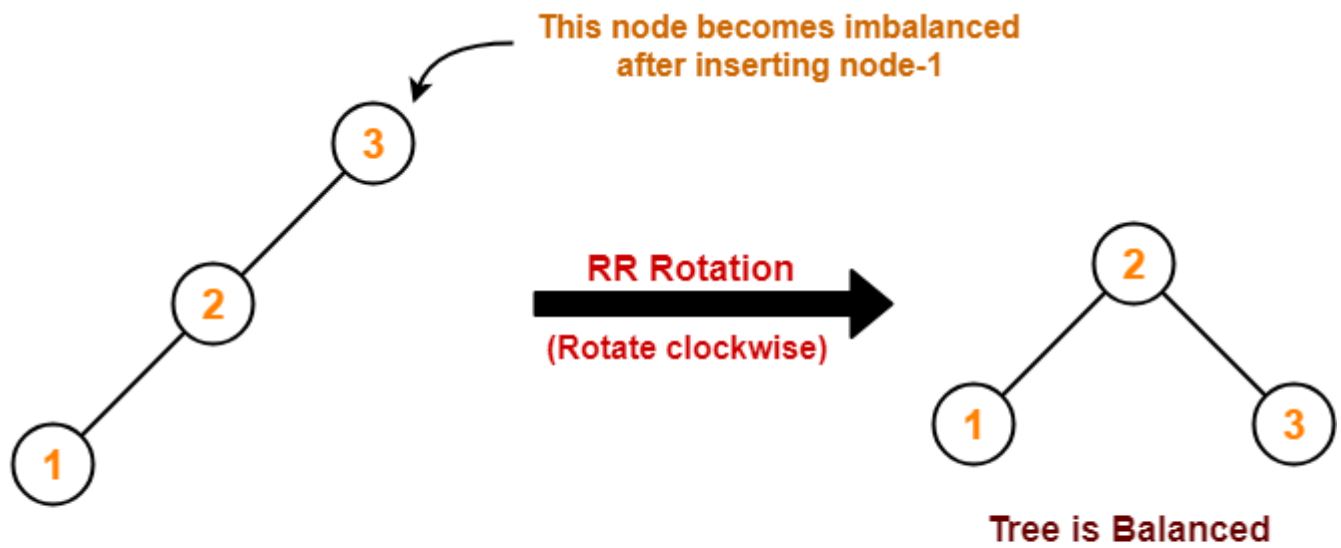
Case-01:



Insertion Order : 1 , 2 , 3

Tree is Imbalanced

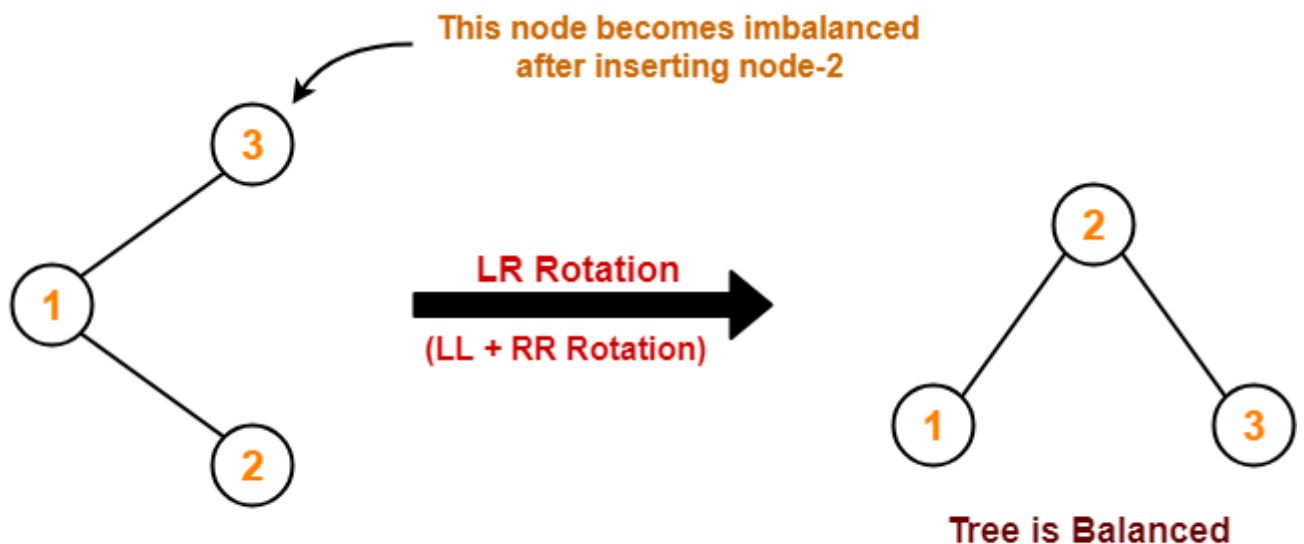
Case-02:



Insertion Order : 3 , 2 , 1

Tree is Imbalanced

Case-03:

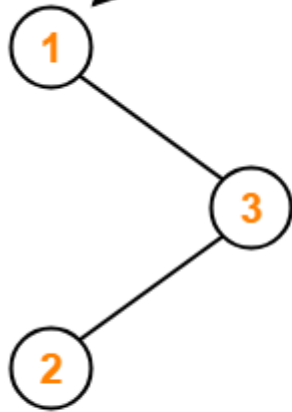


Insertion Order : 3 , 1 , 2

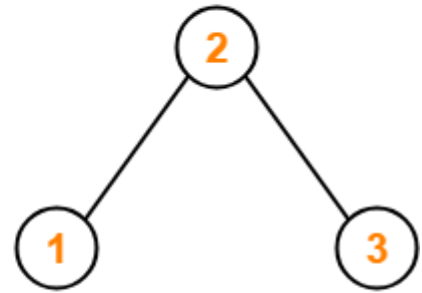
Tree is Imbalanced

Case-04:

This node becomes imbalanced
after inserting node-2



RL Rotation
RR + LL Rotation



Tree is Balanced

Insertion Order : 1 , 3 , 2

Tree is Imbalanced